

GPT-5.5 Direct-Edit Quality Frontier: Six Leaves vs Three Leaves

50 valid runs per cell on RuleLedger v3

By Adam Owada, with Codex
July 1, 2026

Abstract

This paper extends the GPT-5.5 direct-edit quality-frontier study from a four-cell, six-leaf matrix to an eight-cell leaf-count comparison. The original frontier used 50 valid runs per cell for six GPT-5.5 direct-edit leaves. The new expansion adds 50 valid runs per cell for the same root and leaf reasoning pairs, but with three GPT-5.5 direct-edit leaves. In total, the analysis covers 400 valid scored RuleLedger v3 implementations.

The motivating hypothesis was that fewer leaves might help xhigh roots by reducing integration burden. The result does not support that hypothesis as a mean-quality claim. With xhigh roots, three leaves scored 0.7219 mean quality and six leaves scored 0.7249; the three-minus-six delta was -0.0030 with a 95% bootstrap interval of [-0.0488, +0.0421]. Matched xhigh-root comparisons were also unresolved: three leaves were +0.0018 ahead with high leaves and -0.0079 behind with xhigh leaves.

The useful result is different. Three leaves preserved nearly the same xhigh-root quality while spending much less. Across xhigh-root cells, the three-leaf topology used 81.6% as many GPT-5.5 implementation tokens as the six-leaf topology. Across all cells, three leaves used 79.5% as many implementation tokens and improved mean quality-per-token from 1.266e-07 to 1.576e-07.

The highest observed mean-quality cell remains six leaves with a high root and high leaves: 0.7495. The best three-leaf mean was xhigh root with xhigh leaves at 0.7229, closely followed by xhigh root with high leaves at 0.7209. The practical conclusion is therefore: use six-leaf high/high when single-run mean quality is the main goal; use three leaves when token efficiency or lower orchestration load matters; and continue treating xhigh reasoning as an interaction-sensitive knob, not a monotonic quality dial.

Decision Summary

Developer goal	Recommended configuration	Evidence
Best observed mean quality	Six leaves, high root + high leaves	GQF0 mean quality 0.7495, highest of all eight cells
Best three-leaf quality	Three leaves, xhigh root + xhigh leaves	GQ3L2 mean quality 0.7229
Best token efficiency	Three leaves, high root + high leaves	GQ3L1 quality/token 1.810e-07
Best xhigh-root cost reduction	Three leaves with either leaf reasoning	Xhigh-root quality delta -0.0030, token ratio 0.816
Hypothesis outcome	Not supported as a quality uplift	Three leaves did not improve xhigh-root mean quality
Most important caveat	Three-leaf high/high loses quality	GQ3L1 trailed GQF0 by -0.0559 mean quality

Experiment Matrix

All runs used the same frozen RuleLedger v3 starter project, hidden test suite, scoring profile, prompt templates, and staged direct-edit workflow. The judge was GPT-5.5 xhigh for every run. The only experimental changes in the expansion were leaf count and the three-role assignment set.

Cell	Leaf count	Root reasoning	Leaf reasoning	Leaf model	Write mode	Valid runs
GQF0	6	high	high	GPT-5.5	direct edit	50
GQF1	6	xhigh	high	GPT-5.5	direct edit	50
GQF2	6	high	xhigh	GPT-5.5	direct edit	50
GQF3	6	xhigh	xhigh	GPT-5.5	direct edit	50
GQ3L0	3	high	xhigh	GPT-5.5	direct edit	50
GQ3L1	3	high	high	GPT-5.5	direct edit	50
GQ3L2	3	xhigh	xhigh	GPT-5.5	direct edit	50
GQ3L3	3	xhigh	high	GPT-5.5	direct edit	50

The six-leaf topology used the earlier role split: TypeScript parsing and compatibility; TypeScript replay, reporting, and performance; Python parsing and compatibility; Python replay, reporting, and performance; cross-language parity and regression review; and adversarial review.

The three-leaf topology used broader leaves:

1. TypeScript parsing, normalization, replay, billing, reporting, public API compatibility, and performance.
2. Python parsing, normalization, replay, billing, reporting, public API compatibility, and performance.
3. Cross-language parity, regression coverage, hidden-risk review, and integration simplification.

Execution Audit

The 400-run analysis pools three validated experiment directories:

Batch	Directory	Runs per cell	Notes
Six-leaf original	runs/20260629T224027-gpt55_direct_quality_frontier-gpt55_frontier_j7_j6	20	Initial 80-run experiment
Six-leaf extension	runs/20260630T061938-gpt55_direct_quality_frontier_50-gpt55_frontier_50_r21_r50_j7_j6	30	Rerun-repaired 120-run extension
Three-leaf expansion	runs/20260701T051040-gpt55_direct_quality_frontier_3leaf_50-gpt55_frontier_3leaf_50_j7_j6	50	Rerun-repaired 200-run expansion

The three-leaf expansion initially encountered usage-limit contamination. In the first pass, all judges failed, much of GQ3L1 failed, and the GQ3L2/GQ3L3 implementation cells produced usage-limit artifacts. Those zero scores were not treated as signal. Invalid phases were rerun after usage resets until all 200 implementations completed, all 200 judges parsed strict JSON, and Stage 11 validation passed.

The final validation status for the three-leaf experiment is passed. The combined analysis script also passed: exactly 50 rows per cell, no duplicate run IDs, complete artifacts, no preserved failure phases, no usage warnings, no score warnings, and no non-positive quality scores.

The reproducible analysis commands are:

```
python scripts\analyze_gpt55_frontier_50.py
python scripts\analyze_gpt55_leaf_count_frontier_50.py
```

The second command writes derived comparison outputs under runs/analysis/gpt55_direct_quality_frontier_leaf_count_50.

Primary Results

Cell	Configuration	Runs	Mean quality	95% bootstrap CI	Hidden correctness	Judge	GPT-5.5 impl tokens	Quality/token
GQF0	6 leaves, high root + high leaves	50	0.7495	[0.7022, 0.7955]	0.7227	0.7350	5.029M	1.538e-07
GQF1	6 leaves, xhigh root + high leaves	50	0.7190	[0.6770, 0.7629]	0.6974	0.7252	6.213M	1.200e-07
GQF2	6 leaves, high root + xhigh leaves	50	0.6885	[0.6455, 0.7338]	0.6660	0.6516	5.735M	1.222e-07
GQF3	6 leaves, xhigh root + xhigh leaves	50	0.7308	[0.6846, 0.7789]	0.7188	0.7188	6.791M	1.105e-07
GQ3L0	3 leaves, high root + xhigh leaves	50	0.6970	[0.6496, 0.7448]	0.6667	0.6748	4.287M	1.689e-07
GQ3L1	3 leaves, high root + high leaves	50	0.6936	[0.6435, 0.7450]	0.6697	0.6615	3.984M	1.810e-07
GQ3L2	3 leaves, xhigh root + xhigh leaves	50	0.7229	[0.6773, 0.7700]	0.7111	0.7085	5.535M	1.336e-07
GQ3L3	3 leaves, xhigh root + high leaves	50	0.7209	[0.6775, 0.7652]	0.7122	0.7099	5.082M	1.471e-07

The old six-leaf conclusion still matters: GQF0 remains the best observed single-run default. The expansion adds a second lesson: three-leaf xhigh-root cells retain the same mean-quality band as six-leaf xhigh-root cells, but with substantially lower token spend.

Matched Leaf-Count Comparisons

Reasoning pair	3-leaf cell	6-leaf cell	Quality delta	95% bootstrap CI	P(delta > 0)	Token ratio
High root + high leaves	GQ3L1	GQF0	-0.0559	[-0.1257, +0.0130]	0.0542	0.792
High root + xhigh leaves	GQ3L0	GQF2	+0.0085	[-0.0565, +0.0730]	0.6045	0.748
Xhigh root + high leaves	GQ3L3	GQF1	+0.0018	[-0.0594, +0.0621]	0.5277	0.818
Xhigh root + xhigh leaves	GQ3L2	GQF3	-0.0079	[-0.0730, +0.0574]	0.4052	0.815

The matched comparisons show where fewer leaves helped and where they did not. The high/high six-leaf cell lost the most when collapsed to three broad leaves: -0.0559 mean quality. That is not fully resolved by the bootstrap interval, but the probability of a positive three-leaf effect is only 0.0542.

For xhigh roots, the picture is almost flat. Three leaves were barely ahead with high leaves and barely behind with xhigh leaves. Both intervals are wide and centered near zero. This is why the original hypothesis is not supported: the reduced leaf count did not create an xhigh-root quality uplift.

The token result is much less ambiguous. Every three-leaf matched cell used fewer implementation tokens. The three-leaf token ratio ranged from 0.748 to 0.818, meaning roughly 18% to 25% lower GPT-5.5 implementation spend.

Hypothesis Tests

Effect	Quality delta	95% bootstrap CI	P(delta > 0)	Token ratio
3 leaves minus 6 leaves, all cells	-0.0134	[-0.0462, +0.0199]	0.2118	0.795
3 leaves minus 6 leaves, high roots	-0.0237	[-0.0719, +0.0234]	0.1648	0.768
3 leaves minus 6 leaves, xhigh roots	-0.0030	[-0.0488, +0.0421]	0.4485	0.816
3-leaf xhigh roots minus 3-leaf high roots	+0.0266	[-0.0209, +0.0738]	0.8645	1.284

The xhigh-root hypothesis was directionally plausible: fewer leaves should give the root fewer candidate patches to reconcile, less conflicting code to discard, and a simpler final integration decision. But the observed quality lift did not appear. Three-leaf xhigh roots were statistically indistinguishable from six-leaf xhigh roots and slightly lower on observed mean.

The last row is interesting for a different reason. Within the three-leaf matrix, xhigh roots beat high roots by +0.0266 observed mean quality, with probability positive 0.8645. This is not decisive, but it is a cleaner sign that xhigh root effort may become more useful once the leaf set is smaller. The cost remains real: three-leaf xhigh-root cells spent 1.284x as many implementation tokens as three-leaf high-root cells.

Upper Tail

Cell	Top-quartile mean
GQF0	0.9607
GQF1	0.9669
GQF2	0.9142
GQF3	0.9756
GQ3L0	0.9439
GQ3L1	0.9537
GQ3L2	0.9700
GQ3L3	0.9508

The upper-tail result remains friendly to xhigh/xhigh. The best top-quartile mean overall is still six-leaf xhigh/xhigh, GQF3, at 0.9756. The best three-leaf upper tail is GQ3L2 at 0.9700, close enough to matter operationally.

The top individual runs also show that the three-leaf xhigh cells can reach the ceiling. Among the top 15 runs in the 400-run pool, GQ3L2 appears three times and GQ3L3 appears four times. The best single run remains GQF0 r32 at 0.9904, but the three-leaf xhigh cells are not low-ceiling configurations.

Interpretation

This experiment splits "quality" and "coordination cost" more cleanly than the first six-leaf study. More leaves give the root a broader search portfolio. That helps the high/high six-leaf default and probably explains why GQF0 still has the best mean quality. Fewer leaves reduce spend and integration surface, but the broader

three-leaf leaves appear to lose some of the benefit of specialized search.

For xhigh roots, the tradeoff is more attractive. The three-leaf xhigh cells are effectively tied with their six-leaf counterparts on mean quality, while using about one-fifth fewer implementation tokens. That does not prove that fewer leaves "help" xhigh roots in absolute quality, but it does show that six leaves may be unnecessary for xhigh-root quality on this benchmark.

The xhigh-root result also changes how to read the earlier six-leaf paper. The six-leaf GQF1/GQF3 cells were expensive and not clearly better than GQF0. In the three-leaf matrix, xhigh roots look more competitive: GQ3L2 and GQ3L3 are the top two three-leaf means, and the xhigh-root average is 0.0266 above the three-leaf high-root average. This is not enough to dethrone GQF0, but it is enough to justify a follow-up study focused on smaller xhigh-root teams.

Is 50 Runs Per Cell Enough?

Fifty runs per cell is enough to rule out large effects in several places. If three leaves had meaningfully improved xhigh-root mean quality, the matched comparisons should not have landed within one point of zero. It is also enough to show that the token reduction is large and consistent.

It is not enough to settle small quality differences. The xhigh-root three-minus-six interval is $[-0.0488, +0.0421]$, so modest gains or losses remain plausible. Likewise, the three-leaf xhigh-root advantage over three-leaf high roots is suggestive but not conclusive.

The practical reading is:

1. Six-leaf high/high remains the best observed highest-quality default.
2. Three-leaf xhigh-root cells are quality-competitive with six-leaf xhigh-root cells and cheaper.
3. The exact quality ranking of the xhigh-root cells is still too tight for a definitive claim.

Recommendations

For RuleLedger-style direct-edit subagent work:

1. Keep six-leaf high root plus high leaves as the default when the objective is highest expected single-run quality.
2. Use three-leaf xhigh-root cells when token budget or orchestration latency matters and a small possible quality tradeoff is acceptable.
3. Do not interpret "fewer leaves help xhigh roots" as a proven quality law. The evidence supports "fewer leaves preserve xhigh-root quality more cheaply."
4. Avoid three-leaf high/high as a replacement for six-leaf high/high if the goal is maximum quality; that matched comparison had the largest observed loss.
5. Continue studying three-leaf xhigh/xhigh as a peak-run candidate, because its upper tail was close to six-leaf xhigh/xhigh at lower token cost.

Limitations

This is one benchmark and one model family. RuleLedger v3 rewards coherent cross-language semantics, replay correctness, deterministic reporting, and hidden-case robustness. Other tasks may benefit more from broader leaf search or more specialized leaves.

The three-leaf experiment required several usage-limit repair passes. The final data are valid after rerunning invalid phases, and validation confirms no preserved failure phases or unparsed judges. Still, the repair history matters: usage-limit zeros are infrastructure artifacts and must be repaired rather than interpreted.

The bootstrap intervals are empirical intervals over observed runs. They do not guarantee stability across future model snapshots, prompt changes, or harness changes. The judge was held constant at GPT-5.5 xhigh, so judge behavior is not part of the experimental matrix.

Conclusion

The three-leaf expansion answers the original question with a useful "no, but." No, fewer leaves did not improve xhigh-root mean quality. But fewer leaves did preserve xhigh-root quality while cutting implementation token spend by about 18%.

The highest-quality default remains six leaves with a high root and high leaves. The most efficient frontier now includes three-leaf teams, especially when the root is xhigh. In other words, the next optimization target is not simply "more reasoning" or "fewer agents." It is finding the smallest team that gives the root enough diverse, correct material without making integration the dominant source of waste.